



**Università degli Studi di Padova**

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



**Gruppo RubberDuck**

email: [GroupRubberDuck@gmail.com](mailto:GroupRubberDuck@gmail.com)

**Dubbi su Specifica Tecnica**  
**2026-04-30**

## Indice

|                                                       |   |
|-------------------------------------------------------|---|
| 1) Informazioni generali .....                        | 1 |
| 2) Diagrammi delle classi .....                       | 2 |
| 2.1) Dubbi grafici .....                              | 2 |
| 2.2) Outbound Port .....                              | 4 |
| 2.3) Dipendenza semplice tra interfacce .....         | 4 |
| 3) Architettura .....                                 | 6 |
| 3.1) Oggetti di passaggio tra adapter e service ..... | 6 |
| 3.2) Dominio e Snapshot .....                         | 6 |

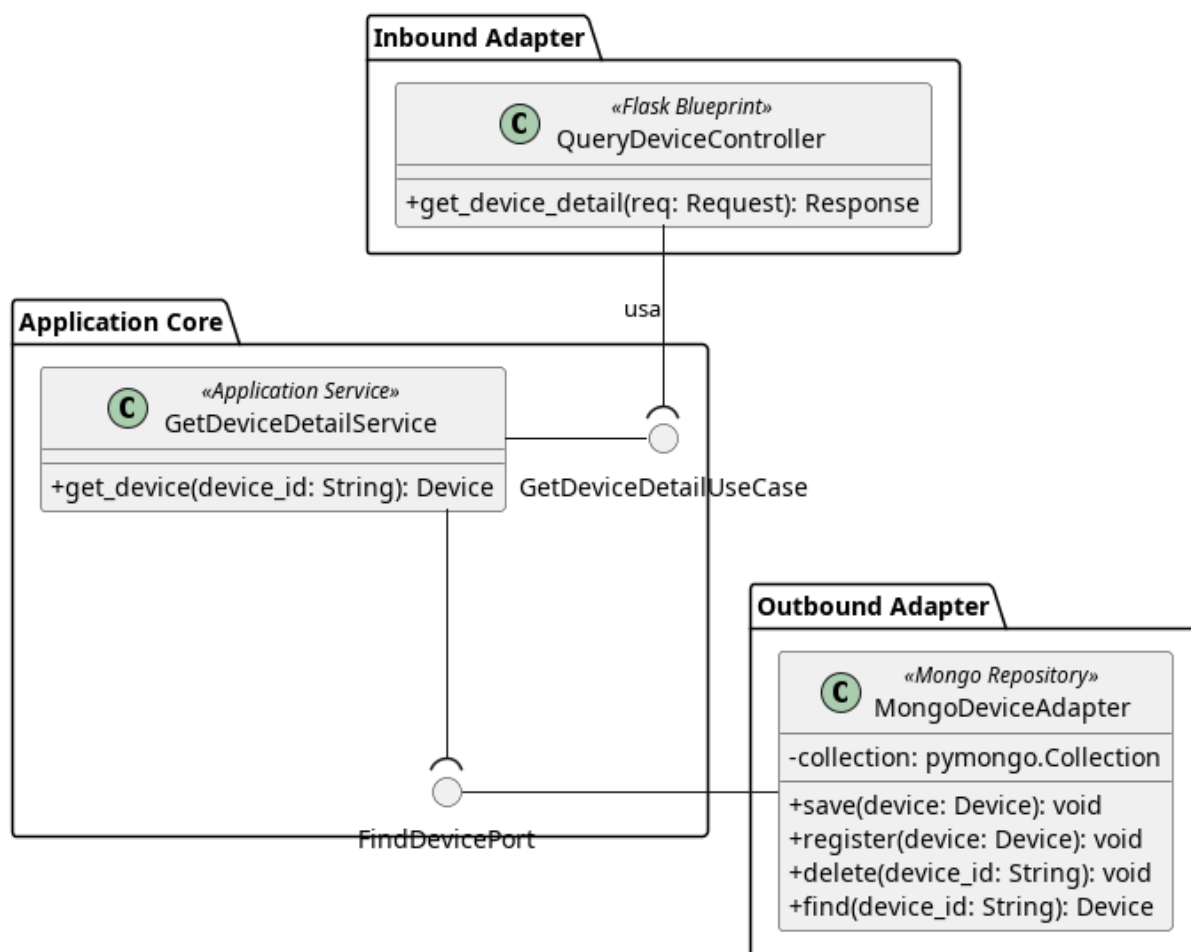
## 1) Informazioni generali

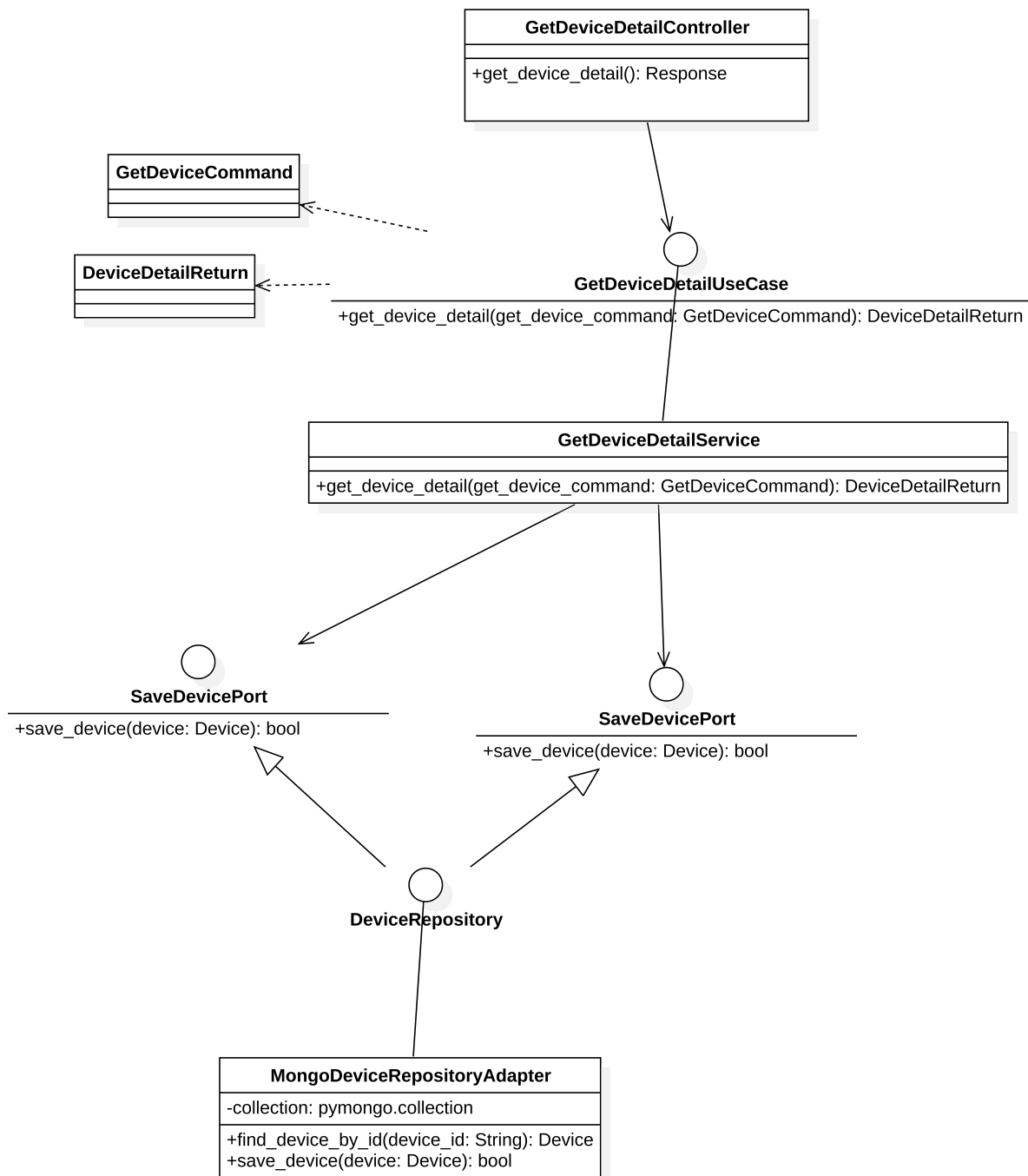
- **Tipo di riunione:** Esterna
- **Motivazione:** Chiarimento dubbi
- **Data:** 2026-04-30
- **Luogo:** Riunione su Meet
- **Ora inizio:** 9.10
- **Ora fine:** 9.30
- **Partecipanti:**
  - Aldo Bettega
  - Davide Lorenzon
  - Riccardo Cardin

## 2) Diagrammi delle classi

### 2.1) Dubbi grafici

A seguito di alcune discussioni interne abbiamo preferito chiedere a lei quale fosse la rappresentazione grafica in UML di alcuni rapporti tra classi

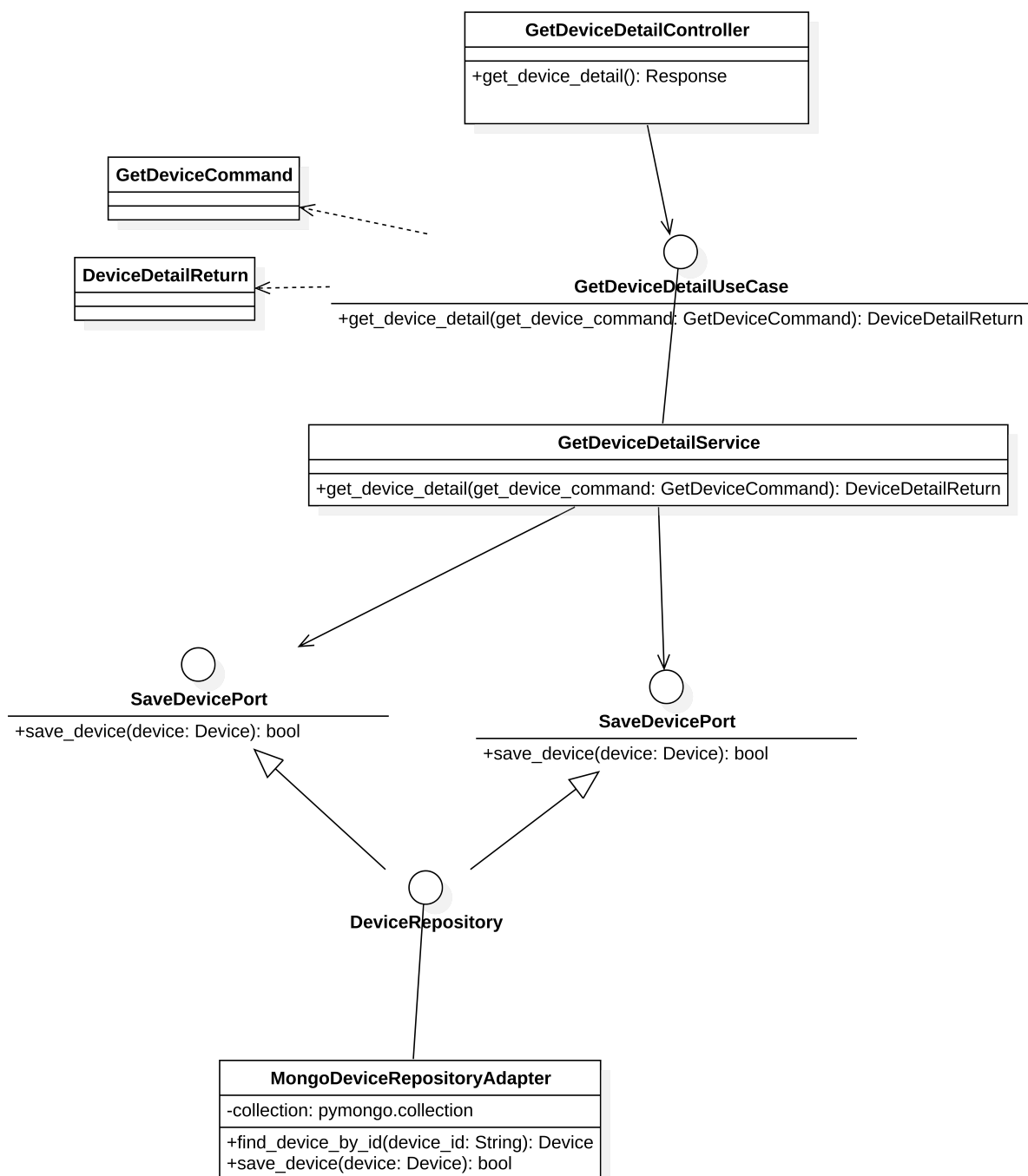




## 2.2) Outbound Port

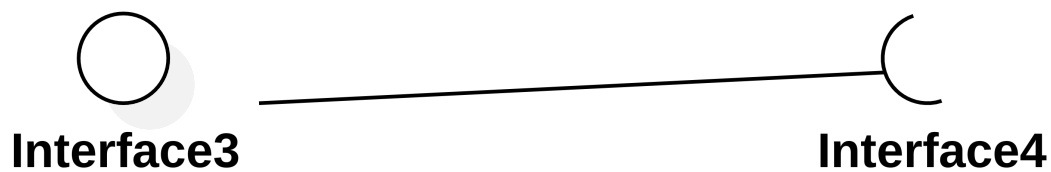
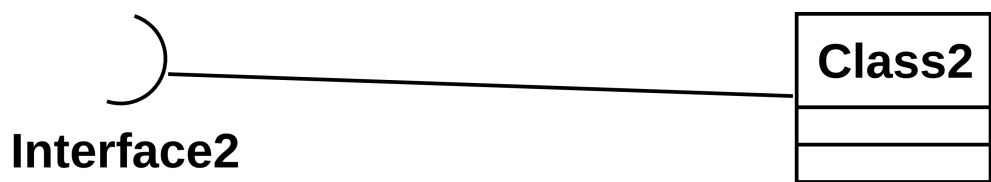
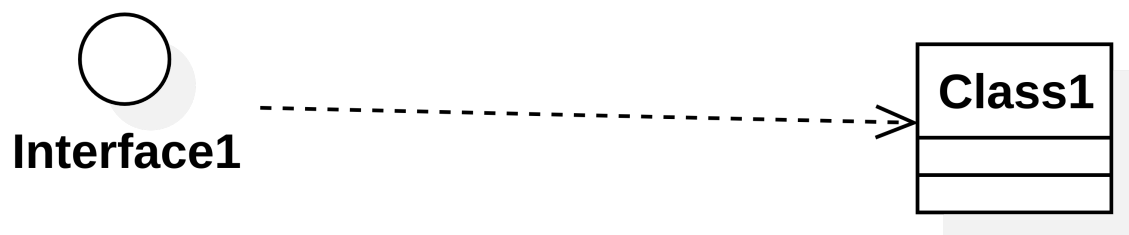
Nella definizione delle outbound port è corretto creare un'interfaccia che eredita da alcune outbound port e poi realizzare un singolo adapter che implementa questa porta, ad esempio le porte che interagiscono con il sistema di permanenza dichiarano ognuna un solo metodo

Poi si aggiunge un'interfaccia che eredita da tutte le interfacce, la chiamiamo repository e poi un ConcreteRepository implementa quest'ultima interfaccia



## 2.3) Dipendenza semplice tra interfacce

Qual è la relazione corretta per rappresentare una dipendenza semplice tra 2 interfacce?



### 3) Architettura

#### 3.1) Oggetti di passaggio tra adapter e service

È giusto usare un oggetto DeviceCommand e DeviceReturn che contiene vari campi dati che passa tra service e adapter? Queste classi non hanno comportamento, hanno solo lo scopo di standardizzare la comunicazione attraverso le porte inbound, ad esempio una porta che richiede la visualizzazione di un asset manda i dati necessari a trovare l'asset come un oggetto getAssetCommand che in questo caso avrà solo un attributo id poi la porta dichiara che ritornerà un oggetto di tipo getAssetResponse contenente tutti dati che dovranno essere mostrati all'utente

#### 3.2) Dominio e Snapshot

Noi all'interno del dominio abbiamo un oggetto device annidato, è composto da Asset che a loro volta sono composti da Answer.

Tuttavia per evitare modifiche che bypassano la classe root e per mantenere l'incapsulamento senza impedire la lettura dei dati dei figli abbiamo introdotto delle classi snapshot delle entità, sono delle copie di sola lettura delle classi possedute dal device.

ci domandavamo se questo approccio fosse accettabile

